

Concepts From Scratch

Eureka, Config Server, and Maven multi-module basics — plus a plain-English walkthrough of every line of code we've actually written so far, in auth-service, user-service, api-gateway, and common.

CONTENTS

1. The 30-second picture
2. Maven basics — pom.xml, parent/child modules, and why `common` exists
3. Eureka — the service phone book
4. Config Server — the shared settings sheet
5. API Gateway — the front desk, briefly
6. auth-service, walked through step by step
7. user-service, walked through step by step
8. The common module, with real code
9. Glossary

Covers: Stage 0-2 of the build

Repo: [Aditya99k/vicinia](#)

Written: 2026-08-21

The 30-second picture

Everything below makes more sense once you have one mental picture to hang it on. Here it is.

Imagine Vicinia is a restaurant with several separate kitchen stations (auth, user, and later merchant, catalog, order...) instead of one giant kitchen. Splitting it up like this is good for scaling and for letting different stations change independently — but it creates three new problems a single kitchen never had:

- **"Where is the grill station right now?"** — stations can move, restart, or multiply, so nobody can just memorize a fixed location. **Eureka** is the receptionist who always knows where every station currently is.
- **"What's today's shared house rule?"** — every station needs the same settings (URLs, secrets, timeouts) without each one keeping its own out-of-date copy taped to the wall. **Config Server** is the one binder with today's rules that every station reads from.
- **"Who do customers actually talk to?"** — a customer shouldn't have to know which station makes what. **api-gateway** is the front desk: customers only ever talk to it, and it quietly routes the request to the right station.

KEEP THIS PICTURE

Receptionist (Eureka) knows where everyone is. One shared rules binder (Config Server) everyone reads from. One front desk (api-gateway) customers actually talk to. Every other concept in this doc is a detail underneath one of those three ideas.

Maven basics – and why we have a **common** module

This is the part almost nobody explains simply. Here it is from zero.

2.1 What even is **pom.xml** ?

Vicinia is written in Java using Spring Boot. Java projects need a build tool to do three jobs: download the libraries your code depends on, compile your code, and package it into a runnable file. **Maven** is that build tool, and **pom.xml** is its instruction file — one per project, listing: what this project is called, what libraries it needs (its *dependencies*), and what Java version to compile against.

2.2 Why we have *multiple* **pom.xml** files

Vicinia isn't one project — it's 16 small services that all happen to live in one repository (a "monorepo"). Each service (**auth-service** , **user-service** , **api-gateway** ...) is its own little Maven project with its own **pom.xml** , living in its own folder under **backend/** .

But they all need the *same* Java version, the *same* Spring Boot version, and the *same* Spring Cloud version — nobody wants 16 services silently drifting onto different library versions over time. So there's one extra file, **backend/pom.xml** , called the **parent POM**. Every service's **pom.xml** points up at it:

BACKEND/AUTH-SERVICE/POM.XML (EXCERPT)

```
<parent>
  <groupId>com.vicinia</groupId>
  <artifactId>vicinia-backend</artifactId>
  <version>0.1.0-SNAPSHOT</version>
</parent>
```

And the parent, in turn, lists every child in a **<modules>** block:

BACKEND/POM.XML (EXCERPT)

```
<modules>
  <module>config-server</module>
  <module>discovery-server</module>
  <module>api-gateway</module>
  <module>common</module>
  <module>auth-service</module>
  <module>user-service</module>
</modules>
```

This is what makes **mvn clean install** , run once from **backend/** , build all six projects in the right order automatically — Maven reads this list and figures out the dependency order itself (e.g. it knows **auth-service** needs **common** built first, because **auth-service** 's own **pom.xml** lists **common** as a dependency).

PLAIN ENGLISH

The parent POM is a company-wide style guide: "everyone uses Java 21, everyone uses this exact Spring Boot version." The `<modules>` list is the company directory: "here are all our departments, build them all when I say go." Each service's own `pom.xml` only needs to say what makes *it* different — which libraries it personally needs.

2.3 Why a `common` module exists

Three services — `auth-service`, `user-service`, and `api-gateway` — all need the exact same JWT-checking logic and the exact same "did this request actually come through the gateway?" check. Copy-pasting that code into all three would mean fixing the same bug in three places later, and risking them silently drifting out of sync.

Instead, that shared code lives in its own tiny module, `backend/common`, which isn't a runnable service at all — it's a library. Any service that needs it just adds one line to its own `pom.xml`:

```
<dependency>
  <groupId>com.vicinia</groupId>
  <artifactId>common</artifactId>
</dependency>
```

What actually lives in `common` is covered properly in §8 — for now, just remember: **one copy of shared code, three services borrowing it**, instead of three copies quietly diverging.

PLAIN ENGLISH

`common` is the shared toolbox on a job site. Every contractor (service) that needs the same drill just grabs it from the shared box instead of everyone buying and maintaining their own slightly-different drill.

Eureka – the service phone book

3.1 The problem, in plain terms

When `api-gateway` needs to forward a request to `auth-service`, what address does it use? You might think "just hardcode `http://localhost:8081`" — and that works, right up until `auth-service` restarts on a different port, moves to a different machine, or you run three copies of it for more traffic. A hardcoded address breaks the instant reality changes underneath it.

3.2 How Eureka actually works

- 1 **A service starts up** and sends Eureka one message: "I'm `AUTH-SERVICE`, and I live at this address."
- 2 **It keeps checking in** — every 30 seconds it pings Eureka with "still here" (called a *heartbeat*).
- 3 **If it goes quiet** for 90 seconds (no heartbeat), Eureka assumes it died and removes it from the list.
- 4 **Anyone who needs to find it** — like `api-gateway` — just asks Eureka "who is `AUTH-SERVICE` right now?" instead of hardcoding an answer.

3.3 Our actual config, line by line

BACKEND/DISCOVERY-SERVER/SRC/MAIN/RESOURCES/APPLICATION.YML

```
server:
  port: 8761                # the phone book itself lives here

eureka:
  client:
    register-with-eureka: false  # the phone book doesn't list itself
    fetch-registry: false
  server:
    enable-self-preservation: false  # dev-only: evict fast, don't linger on stale entries
    eviction-interval-timer-in-ms: 5000
```

`enable-self-preservation: false` is a deliberate dev-only choice: in production, Eureka protects itself from over-reacting to what might just be a network hiccup by refusing to mass-evict everyone at once. Locally, we'd rather a restarted service disappear from the list in ~5 seconds than sit there looking "alive" for up to 90.

Config Server – the shared settings sheet

4.1 The problem, in plain terms

Several services need to know the exact same secret values — for example, the JWT signing key has to be *identical* in `auth-service` (which creates tokens) and `api-gateway` (which checks them), or nothing would ever verify. Copy-pasting that secret into six different `application.yml` files means the day it needs to change, you have to remember to update all six, correctly, everywhere.

4.2 How it works

Config Server is a tiny web service with exactly one job: when asked "what's the config for `auth-service`?", it reads a file and hands back the answer as JSON. Every other service asks it this one question on startup, instead of keeping its own copy of shared settings.

BACKEND/CONFIG-SERVER/SRC/MAIN/RESOURCES/APPLICATION.YML

```
spring:
  profiles:
    active: native           # read config from local files (simplest option)
  cloud:
    config:
      server:
        native:
          search-locations: classpath:/config-repo
```

The actual shared values live in one file, served to every service that asks:

BACKEND/CONFIG-SERVER/SRC/MAIN/RESOURCES/CONFIG-REPO/APPLICATION.YML (EXCERPT)

```
vicinia:
  internal-secret: TznLN0T9...    # shared by every service's gateway-only check
  jwt:
    secret: kxFaHVscTq7...       # shared by auth-service and api-gateway
    access-token-ttl-minutes: 15
```

And any service that wants these values declares it in its own tiny config, with one important word:

E.G. BACKEND/AUTH-SERVICE/SRC/MAIN/RESOURCES/APPLICATION.YML

```
spring:
  config:
    import: optional:configserver:http://localhost:8888
```

The word `optional:` matters: without it, a service **refuses to start at all** if Config Server happens to be down. With it, the service just falls back to whatever's in its own local file — a much friendlier default while Config Server is still simple, dev-only infrastructure.

PLAIN ENGLISH

Config Server is the one binder of "today's shared rules" sitting at the front desk. Every station reads from the same binder instead of keeping its own sticky note — so when a rule changes, you update the binder once, not six sticky notes you might forget about.

API Gateway – the front desk, briefly

Customers (browsers, apps) only ever talk to `api-gateway`, never directly to `auth-service` or `user-service`. Two jobs happen here, on every single request, before it's forwarded anywhere:

- 1 **Routing** — a request to `/api/auth/**` gets forwarded to whichever `AUTH-SERVICE` instance Eureka currently says is alive.
- 2 **Security** — unless the path is on a public allow-list (like `/api/auth/login`), the gateway requires a valid access token, checks it hasn't been logged-out/blacklisted, and then tells the downstream service *who's calling* via a header — the downstream service never has to check the token itself, it just trusts the gateway.

This is covered in much more depth (with diagrams) in `study/2026-08-20-infra-fundamentals.html` — this section is intentionally short since that material already exists.

auth-service, step by step

auth-service owns exactly one job: proving who someone is, and handing them a token that proves it to everyone else.

6.1 The database tables, in plain terms

TABLE	WHAT IT ACTUALLY HOLDS
<code>user_credentials</code>	One row per person: their email and their <i>hashed</i> password (never the real password — see below).
<code>roles</code>	The 4 job titles: CUSTOMER, MERCHANT, DELIVERY_PARTNER, ADMIN.
<code>permissions</code>	The individual things someone might be allowed to do, like <code>ORDER_PLACE</code> or <code>MERCHANT_APPROVE</code> .
<code>role_permissions</code> , <code>user_roles</code>	Just the "who's connected to what" link tables — a role links to many permissions, a person links to (usually one) role.

PLAIN ENGLISH

Think of a role as a lanyard badge, and permissions as the individual doors it opens. Instead of hardcoding "if badge color is blue, let them into the server room," we keep an actual list of which doors each badge color opens — so adding a new badge type later is just adding a new list, not rewriting every door.

6.2 What a "hashed" password actually means

We never store your actual password. When you sign up, the password goes through a one-way scrambling function called **bcrypt** — one-way meaning there's no way to un-scramble it back to the original. When you log in later, we scramble what you typed the same way and check if the scrambled versions match. If our database ever leaked, nobody could read anyone's actual password from it.

6.3 What a JWT (the "access token") actually is

Think of it like a wristband at a concert. Once you're stamped at the entrance, security anywhere inside just glances at the wristband — they don't re-check your ID at every door. A JWT is that wristband: a piece of text that says "this is user X, with these roles and permissions," *cryptographically signed* so nobody can forge or edit it without the signing having a secret key. It expires after 15 minutes, same as a wristband only being valid for tonight's show.

6.4 What a "refresh token" is, and why it's different

If the wristband (access token) expires every 15 minutes, you'd have to log in again constantly — annoying. So on login you also get a **refresh token**: think of it like a coat-check ticket. It doesn't get

you into the concert itself, but you can trade it in for a brand-new wristband without showing ID again. Unlike the wristband, the coat-check ticket is stored in our system (Redis) so we can invalidate it — cancel someone's coat-check ticket and they can't get a new wristband anymore, even though their old wristband might still be technically valid for a few more minutes.

6.5 Signing up, step by step

- 1 Check the email isn't already registered.
- 2 Reject if someone tried to sign up directly as **ADMIN** — that role can only be granted by an existing admin, never self-service.
- 3 Scramble the password with bcrypt, save the new row.
- 4 Announce "a new user signed up" onto Kafka — a bulletin board other services (like user-service) are watching. See §7.
- 5 Hand back a wristband (access token) and a coat-check ticket (refresh token), exactly like a login would.

BACKEND/AUTH-SERVICE/.../AUTHSERVICE.JAVA (SIMPLIFIED)

```
public AuthResponse signup(SignupRequest request) {
    if (userCredentialRepository.existsByEmail(request.email())) {
        throw new EmailAlreadyExistsException(request.email());
    }
    // ...role validation...
    UserCredential user = new UserCredential(request.email(),
        passwordEncoder.encode(request.password())); // bcrypt scramble
    user.addRole(role);
    user = userCredentialRepository.save(user);

    userEventPublisher.publishUserRegistered(...); // post to the bulletin board

    return issueTokens(user); // hand back the wristband +
    ticket
}
```

6.6 Logging out, step by step

Logging out has to actually revoke a wristband that hasn't naturally expired yet — a signature alone can't express "this one, specifically, no longer." So we keep a small "banned wristbands" list in Redis:

- 1 Read the wristband (access token) straight from the request itself.
- 2 Add its unique ID to the "banned" list in Redis, set to auto-expire at exactly the same time the wristband would've expired anyway — no point banning it forever.
- 3 Also cancel their coat-check ticket (refresh token), so they can't quietly get a new wristband either.

From then on, every request through `api-gateway` checks that "banned" list before trusting any wristband — that's the one thing a signature alone could never do.

user-service, step by step

7.1 What Kafka actually is, in plain terms

Kafka (Redpanda, in our case — a compatible drop-in) is a shared bulletin board. A service can pin a note to it ("user X just signed up!") without knowing or caring who's going to read that note, or when. Other services can walk past the board whenever they like and react to notes that matter to them. This is different from a phone call (like our gateway-to-service calls) where you need the other person to answer *right now*.

7.2 Why there's no "create profile" endpoint

You'll notice `user-service` has no `POST /users` — you can't directly ask it to create a profile. Instead, it's constantly watching the bulletin board (the `user-events` Kafka topic), and the moment it sees a "user registered" note pinned by `auth-service`, it creates the matching profile row itself:

BACKEND/USER-SERVICE/.../USEREVENTCONSUMER.JAVA (SIMPLIFIED)

```
@KafkaListener(topics = "user-events", groupId = "user-service")
public void onUserEvent(UserEventEnvelope envelope) {
    if (envelope.eventType().equals("user.registered")) {
        String userId = (String) envelope.payload().get("userId");
        if (!userRepository.existsById(UUID.fromString(userId))) {
            userRepository.save(new UserProfile(...));
        }
    }
}
```

7.3 The "did I already do this?" check, explained

Kafka can occasionally deliver the same note twice (it guarantees "at least once," not "exactly once"). If we blindly created a profile every time we saw the note, a duplicate delivery could crash the app (same ID twice) or, in a worse scenario elsewhere in the system, could double-charge someone's account. The fix is simple: before doing anything, check if it's already done.

`existsById(...)` above is exactly that check — "do I already have a profile for this person? If so, do nothing." This pattern is called **idempotency**, and it shows up everywhere a note might arrive more than once.

The **common** module, with real code

Three small, focused pieces of code live here, used by more than one service:

8.1 **JwtTokenProvider** – makes and checks wristbands

Used by **auth-service** to *create* wristbands, and by **api-gateway** to *check* them. Both need the exact same secret key and logic, or a wristband created by one wouldn't be recognized as valid by the other — which is exactly why this lives in one shared place instead of two separate (and easy-to-mismatch) copies.

8.2 **InternalRequestFilter** – the "did you come through the front desk?" check

Every service that receives requests (**auth-service** , **user-service** , and every future one) needs to reject anyone hitting its own port *directly*, bypassing **api-gateway** entirely. The gateway stamps a shared secret header on every request it forwards; this filter checks for that exact stamp and rejects anything missing it.

BACKEND/COMMON/.../INTERNALREQUESTFILTER.JAVA (SIMPLIFIED)

```
String provided = request.getHeader("X-Internal-Secret");
if (provided == null || !provided.equals(expectedSecret)) {
    response.sendError(403, "Direct access is not permitted – go through api-gateway");
}
```

8.3 **EventEnvelope** – the standard shape for a bulletin-board note

Every note posted to Kafka, no matter which service posts it or what it's about, has the same basic wrapper: a unique ID (so a duplicate delivery can be spotted), what kind of note it is, and when it happened. Standardizing that shape once, in **common** , means every future service posts notes the same way instead of everyone inventing their own format.

Glossary

TERM	PLAIN-ENGLISH MEANING
JWT	A signed, tamper-proof piece of text proving who you are — the "wristband."
Refresh token	A revocable ticket you trade in for a new JWT without logging in again — the "coat-check ticket."
RBAC	Role-Based Access Control — deciding what someone can do based on their role's list of permissions, not a single hardcoded check.
bcrypt	A one-way scrambling function for passwords — can't be reversed, only compared against.
Idempotent	Safe to do more than once by accident — the second attempt is a no-op instead of a duplicate/error.
Kafka / Redpanda	A shared bulletin board services post notes to and read from, without calling each other directly.
Eureka	The receptionist that always knows every service's current address.
Config Server	The one shared binder of settings every service reads from at startup.
api-gateway	The front desk — the only thing outside callers ever talk to directly.
DTO	Data Transfer Object — a small class that's just a shape for request/response data, no behavior.
Repository	A Spring Data interface that reads/writes a database table without you writing SQL by hand.
Bean	Spring's word for "an object it manages for you" — created once, handed to whatever needs it.
POM / <code>pom.xml</code>	Maven's instruction file: what this project is, and what it depends on.
Parent POM	The shared instruction file every service's own <code>pom.xml</code> points up at, for consistent versions.